From Event Memory to Physical World State: A Position on the Missing Variable in Robot World Models

Zhang Jie*
Independent Researcher

July 2026

Abstract

World Action Models (WAMs) have recently emerged as a promising paradigm for long-horizon robot manipulation, compressing interaction history into event memories to guide next-subtask prediction. We argue that this paradigm rests on a missing architectural variable: a *persistent, revisable, physically-constrained world state*. Analyzing the frontier of event-memory-based WAMs, we identify four systematic failure modes—*sealed errors*, *salience-driven forgetting*, *repeated re-derivation*, and *error entrenchment*—that arise because state is re-derived each step from an append-only memory rather than maintained as an independent variable. We further argue that three properties often cited as requirements for such a state (persistence, observational revisability, physical consistency) are necessary but insufficient; a fourth property—*counterfactual simulability*—is both essential and the most expensive to obtain. This leads us to a re-positioning of world models themselves: rather than serving as the robot's state representation, world models should model the *transition kernel* over an independently-maintained state, i.e., the distribution $P(s' \mid s, a)$, while the current value of s is owned by a separate state-tracking mechanism. We call this decoupled architecture the *Physical World State* (PWS) framework, and we show that it clarifies several open problems in embodied AI, including goal-revision during task execution, sim-to-real transfer in safety-critical settings, and the structural reasons why current systems excel in structured industrial environments yet remain fragile in homes and other unstructured domains. We close with a research agenda and a discussion of the industrial implications, arguing that the absence of PWS—not benchmark scores—is the binding constraint on the next phase of embodied AI deployment.

Keywords: world models, embodied AI, robot manipulation, state estimation, long-horizon planning, world action models, position paper

1 Introduction: The Cup on the Table

A robot places a cup on a table and turns to open the door. To a human observer, nothing has "happened"—no new action, no new event. Yet in the physical world, everything has changed: the cup has transitioned from *grasped* to *supported*, and the robot's hand has transitioned from *occupied* to *free*. It is precisely these two changes, taken together, that make "open the door" feasible.

This mundane instant exposes a gap at the heart of contemporary robot world models. The field has made remarkable progress in compressing long interaction histories—extending context windows, distilling events into memory tokens, retrieving relevant episodes—but it has largely avoided a more fundamental question: *what is the robot's persistent representation of the current state of the world, and who owns it?*

*Independent Researcher. Position paper prepared for discussion. The analysis targets architectural paradigms rather than any specific company or product.

Recent work on World Action Models (WAMs) represents the frontier of the compression approach [8, 9, 10, 11, 16, 17]. Rather than lengthening context, these systems compress history into compact event memories and use them to predict the next atomic subtask. The results are impressive on standard benchmarks [41, 42, 44], and the paradigm is attracting significant industrial investment, particularly among a class of *world-model-native* companies that position the world model itself as the robot's pretrained foundation, to be bound to specific embodiments with minimal in-context data.

This paper is a position, not a survey and not a benchmark report. Our claim is sharp:

Central claim

The binding constraint on long-horizon embodied intelligence is not memory capacity, retrieval quality, or even world-model fidelity. It is the absence of an independently-maintained **Physical World State** (PWS): a persistent, revisable, physically-constrained, and counterfactually-simulable variable that represents *what is true now*, distinct from the memory of *what has happened*. Event-memory-based WAMs, by re-deriving state from an append-only log at every step, exhibit four systematic failure modes that become fatal precisely in the unstructured, safety-critical environments—homes, hospitals, hazardous sites—where embodied AI promises the most value.

We develop this claim in four steps. Section 2 gives a fair anatomical reading of the event-memory WAM paradigm, identifying exactly what it compresses and what it loses. Section 3 presents the four failure modes as structural consequences of "state-as-recomputation." Section 4 introduces the PWS framework, including our proposed fourth property (counterfactual simulability) and the re-positioning of world models as transition kernels rather than state representations. Section 5 discusses industrial implications, including why current commercial deployments succeed in structured settings and what the PWS lens predicts about the industry's near-term evolution.

Scope and stance. Our critique is architectural, not corporate. We analyze a paradigm exemplified by recent frontier systems—including the WorldScape line of work [7, 8, 9] and related object-addressable approaches [10, 27, 28]—but our argument applies to any system in which state is recomputed from an append-only memory rather than maintained as an independent variable. Where we reference industrial deployments, we anonymize specific companies to keep the focus on paradigms rather than products. We have also engaged directly with authors of frontier WAM systems; where relevant, we note which of our questions remain unanswered.

2 Anatomy of the Event-Memory Paradigm

To be fair to the paradigm we critique, we must first state precisely what it achieves and why it was a genuine advance.

2.1 The problem: visual self-similarity across task stages

Long-horizon robot manipulation suffers from a well-known pathology: *different stages of a task can look identical* [47, 45, 11]. The tabletop before a subgoal is completed and after it is completed may differ only in ways invisible to current perception—yet the correct next action depends entirely on what has already been accomplished. Extending the context window does not solve this; it merely delays the point at which the model must decide what to attend to.

2.2 The WAM solution: history → task progress

The World Action Model paradigm addresses this by making compression an explicit architectural objective [8, 9]. Interaction history is distilled into gist tokens; event boundaries are detected via cosine-distance change detection; salient anchor points are retained; a global history view is fused with the

task instruction; and the result is retrieved to condition a vision-language model's prediction of the next subtask [9, 11, 12].

What the paradigm got right

For the first time, *compressing history*—rather than *lengthening context*—was made an explicit architectural goal. The demonstration that a compressed "task progress" representation can outperform raw long-context approaches is a real contribution, and the engineering is solid.

The critical question, however, is not whether history was compressed, but *what the compression produced*. We argue it produced a progress *estimate*, not a progress *state*—and the difference is architectural, not cosmetic.

2.3 Two structural observations

Observation 1: "Task progress" is an unordered average of history. In the frontier formulation we examined [9], the global-history component is computed by mean-pooling all historical tokens and fusing the result with the task instruction. Mean pooling destroys order: "place the cup, then open the door" and "open the door, then place the cup" become nearly indistinguishable after averaging. The core concept of "which step we are on" is therefore not carried by any explicit state variable; it is *guessed anew at every step* from an orderless summary.

Observation 2: "Autonomous planning" is per-second prediction. The planning prompt is a fixed template: given the task instruction and current observation, predict the *next second*'s atomic subtask. What is called long-horizon planning is, in fact, short-horizon prediction iterated across a memory scaffold. The system's effective horizon is one second; its long-horizon competence is entirely mortgaged to the reliability of memory retrieval.

The structural conclusion

In such systems, **state is not a maintained variable but a recomputed artifact**: at each step, it is reconstructed from an append-only memory. Memory is the system's only representation of the world, and memory is not state.

This is the crux. The remainder of the paper explores why this mechanism is fragile, what would replace it, and why the replacement is harder than it looks.

3 Four Systematic Failure Modes

If state is recomputed each step from an append-only memory, four failure modes follow with structural necessity. We name them to make them discussable.

3.1 Failure Mode 1: Sealed errors

Each historical record is encoded and frozen at the moment it is written. If that encoding is wrong—due to occlusion, lighting, viewpoint, or simple perception error—the erroneous record *remains in memory forever*, participating in every future retrieval. When later observations reveal that "the cup was not, in fact, placed stably," the system cannot revise the old record; it can only append a new one, leaving two contradictory records to compete for attention.

True state tracking performs belief revision: new evidence overwrites old belief [36, 37]. Append-only logs have "write" and "retrieve," but no "revise." The longer the episode, the more sealed errors accumulate, and the more retrieval becomes a negotiation among contradictions.

3.2 Failure Mode 2: Salience-driven forgetting

This is the deepest failure mode, because it arises from a systematic mismatch between two ordering functions. Event boundaries are selected by cosine-change magnitude: what is retained depends on *how much the observation appears to change*. But the physical states that determine what actions are feasible—whether the hand is occupied, whether the cup is stable, whether the door is latched—are often *visually uneventful*.

The consequence is a perverse selection pressure: slow, continuous physical changes (a gradually loosening gripper) produce no perceptual jump, so the true state-transition points receive no anchor; meanwhile, camera jitter or a passerby generates visual spikes that crowd genuinely important old anchors out of the retrieval budget.

Direct exchange with the authors of a frontier system [54] provides a striking confirmation of this asymmetry. During *training*, event boundaries are generated from embodiment signals—pose velocity, gripper state—and only then annotated semantically. During *online execution*, however, boundary selection falls back to cosine change between adjacent VLM chunks: a purely visual signal. In other words, the training pipeline knows what physically matters (the gripper opening), but the deployed system can only guess at it by how much the observation appears to change. The training-deployment gap is, quite literally, Failure Mode 2 baked into the architecture.

The mismatch

Memory orders history by *salience*; action demands state by *causality*. The two orderings differ, and the bias is systematic: the paradigm structurally under-weights exactly the information that is most physically consequential.

3.3 Failure Mode 3: Repeated re-derivation

Because state is re-fused from the memory pool at every step, the same world can yield inconsistent subgoals across adjacent steps if retrieval surfaces different evidence; visually similar historical fragments cross-contaminate each other; and in novel situations—precisely where history is most needed—a learned gating mechanism may shut memory off entirely.

The value of a state variable is precisely its persistence: it promises that *unless contrary evidence appears, the fact remains true*. Retrieval offers no such promise. Each step's world is a fresh reconstruction, and reconstructions of the same world need not agree with each other.

3.4 Failure Mode 4: Error entrenchment

Per-second planning means each subgoal depends on the fragile reconstructed state. Derive the wrong subgoal → execute the wrong action → the world is *physically* changed to match the erroneous state → subsequent retrieval now faces a world in which the error has been *made true*. Errors do not merely propagate; they are entrenched by the physical world itself.

This is the point at which embodied AI diverges sharply from language modeling. In an LLM, an erroneous token pollutes the context but not the world; in a robot, an erroneous action pollutes the world, and the polluted world then feeds back as observation. The cost of Failure Mode 1 (sealed errors) is thus multiplied: not only is the error remembered, it may have been physically enacted.

3.5 Where the critique does not apply

Fairness requires boundaries. Short-term physical dynamics are *not* handled by this memory mechanism—real-time observation streams handle contact changes and object motion, and it is only long-horizon task state that is fragile. Redundancy (full compressed history remaining in the retrieval pool) mitigates single-point failure. Within the evaluation distributions of the papers in question, the mechanism demonstrably works.

But notice the shape of the claimed use case: *long-horizon tasks with inter-stage visual self-similarity*. This is precisely the regime in which the mechanism is structurally weakest. The paradigm's stated target is the paradigm's own blind spot.

Notebook vs. whiteboard

Memory is a notebook: you write forward only, and recover facts by flipping pages; a mis-written page remains forever, and which page you land on depends on retrieval. State is a whiteboard: it shows what is true *now*, and old facts are erased. A notebook, however well-kept, is not a whiteboard—**memory organizes information by "what is worth revisiting," while physical state must answer "what is true right now."**

4 The Physical World State Framework

We now turn from critique to proposal. What would a system look like that maintains state as a first-class variable?

4.1 Three properties widely acknowledged

The literature points to three requirements, which we endorse but argue are insufficient:

1. **Persistent maintenance, not per-step reconstruction.** State is a durable variable that promises consistency; memory is its *input*, not its *carrier*.
2. **Observational revisability.** New evidence can overwrite old state—"believed to be placed stably" can be superseded by "observed to have fallen"—rather than leaving two contradictory records in permanent competition.
3. **Physical-consistency checking.** Retrieved state is validated against physical constraints: a hand cannot be simultaneously occupied and free; a cup cannot be simultaneously grasped and on the table.

A system satisfying these three properties changes the learning objective from

$$\text{instruction} \longrightarrow \text{action}$$

to

$$\text{current state} \longrightarrow \text{goal state} \longrightarrow \text{state transition} \longrightarrow \text{motor skill}.$$

Action becomes merely the instrument of world-change; intelligence becomes the understanding of how states evolve.

4.2 A fourth, more expensive property: counterfactual simulability

We argue these three properties are necessary but not sufficient. A state that only records the present cannot support planning; planning requires asking *"what would the world become if I did A, versus B?"* This is what distinguishes a world model from a state tracker [1, 2, 3, 4].

Property 4: Counterfactual simulability

The Physical World State must support *rollout*: given the current state s and a candidate action a, the system must be able to simulate the distribution over successor states s'. A state that cannot be rolled forward is a database, not a world model.

This fourth property is the most expensive, and it forces a clarification of what world models are *for*.

4.3 From properties to a concrete sketch

Properties without a sketch remain slogans. We therefore offer a minimal, deliberately conservative concrete form, not as a final design but as an existence proof that PWS is implementable with current technology.

State representation: object slots with relational predicates. The state s is a set of *object slots*, each carrying typed attributes, plus a set of *relational predicates* over slots. For the cup-and-door example:

Slot / Predicate	Domain
`hand.status`	{`free`, `holding`(x)}
`cup.support`	{`table`, `hand`, `air`, `shelf`}
`cup.stable`	{`true`, `false`, `unknown`}
`door.latch`	{`latched`, `unlatched`}
`on(cup, table)`	{`true`, `false`}

Two design choices matter. First, attributes have *closed domains*, so a revision is a value assignment, not a free-form edit. Second, an `unknown` value is first-class: the tracker is allowed to say "I do not know whether the cup is stable," which a mean-pooled memory cannot express. Persistence is the default: a slot retains its value until evidence forces a change.

Revision mechanism: evidence → proposal → check → commit. Observations do not write to the state directly. Each new observation produces *evidence*, which is turned into a *proposal* (a set of slot updates); the proposal is passed through the physical-consistency layer; only a proposal that passes is *committed*. A failing proposal is rejected and logged—which itself becomes evidence ("my last action did not have the expected effect"), triggering recovery rather than silent corruption of the state.

Revision loop (pseudocode)

```
state  = init_slots()                    # persistent across steps
memory = []                              # append-only, as input only
for t in each step:
    obs    = perceive()
    memory.append(obs)
    ev     = extract_evidence(obs, state)        # e.g. "cup tilted 40 deg"
    prop   = propose_updates(ev, state)          # e.g. cup.stable := false
    ok     = consistency_check(prop, state)      # hard physical rules
    if ok:
        state.commit(prop)                       # revise: overwrite old value
    else:
        state.flag_conflict(prop); recover()     # never keep contradictions
    goal   = task_index(state, instruction)      # query, not re-derivation
    a      = plan(state, goal, world_model)      # uses P(s'|s,a) rollouts
    execute(a)
```

Three features of this loop deserve emphasis. (i) *Memory is an input, not a carrier*: the notebook may grow forever; the whiteboard does not. (ii) *Revision is overwriting, not appending*: Failure Mode 1 is removed by construction. (iii) *Goal changes are re-queries*: `task_index` maps the (possibly new) instruction onto the *same* persistent slots, so a mid-task goal change preserves valid object state (Section 4.4).

Consistency layer: hard, non-differentiable, independently testable. The layer encodes invariants such as *mutual exclusivity* (`hand.status = free` excludes `holding`(x)), *support transitivity* (if `cup.support=hand` then `hand.status=holding(cup)`), and *action preconditions* (`open(door)` requires `hand.status=free` and `door.latch=unlatched`). These are classical, hand-writable rules—deliberately so. They require

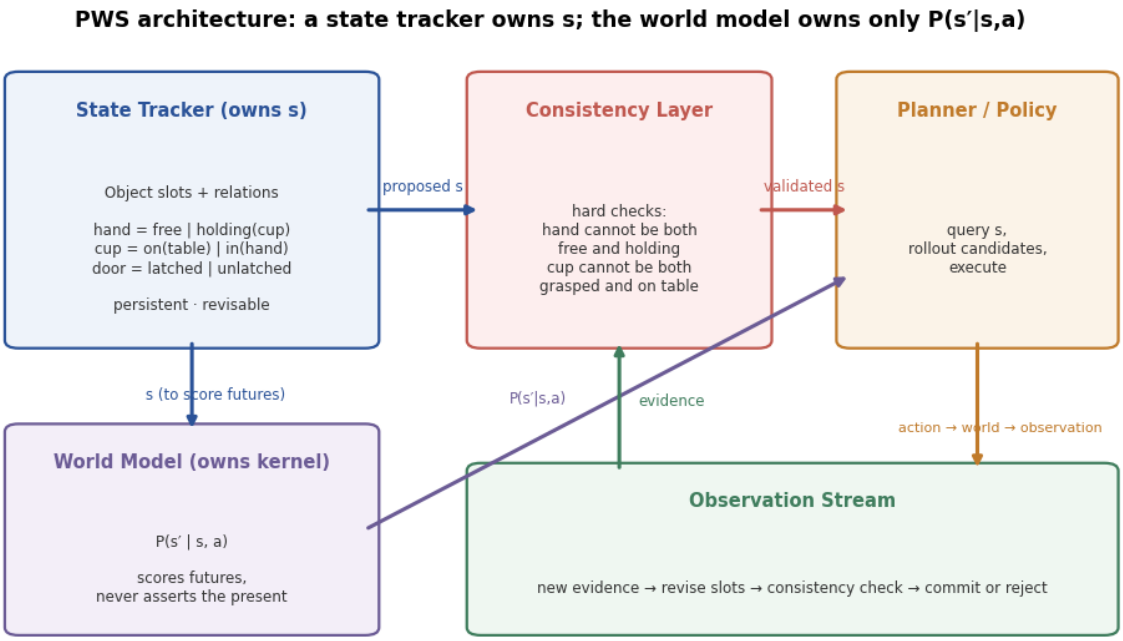

Figure 1: The PWS architecture. A persistent state tracker owns the current state s as object slots with relational predicates; observations revise s only through a hard physical-consistency layer; the world model supplies the transition kernel $P(s' \mid s, a)$ used by the planner to score candidate futures. Memory (not shown) is an append-only input to evidence extraction, never the carrier of state.

no learning, can be unit-tested in isolation, and provide the hard substrate that learned soft constraints (the world model) cannot supply.

Coupling to simulators: the transition kernel is borrowed, not rebuilt. For the rollout property (Section 4.2), the world model need not relearn physics from pixels. Candidate transitions can be scored by coupling the validated state to existing high-fidelity simulators (e.g., physics engines for rigid-body feasibility) for the near-term, geometric part of the transition, and to learned video/latent world models for the appearance-level, long-horizon part. The tracker owns s; the simulators and learned models together approximate $P(s' \mid s, a)$; the consistency layer audits both. Each component is replaceable—this modularity is a feature, not a compromise.

4.4 Re-positioning the world model: from state to transition kernel

The present confusion, we submit, is that frontier systems use the world model to do two jobs at once—represent the current state and predict its evolution—and perform the first job badly because they treat it as an inference over memory. Our proposal is to decouple the jobs:

- A **state tracker** owns the current value of s. It is persistent, revisable, and physically constrained (Properties 1–3). It is updated by observation and corrected by new evidence. It is, in the language of classical AI, a belief state—but one with a hard physical-consistency layer.
- The **world model** owns the transition kernel $P(s' \mid s, a)$. It does not assert what is true now; it predicts what *would* become true *if*. Its proper role is not to *be* the state but to *score the futures* of a state owned elsewhere.

The decoupling

World models should model the *transition kernel* over an independently-maintained state. The current value of s belongs to a state tracker satisfying Properties 1–3; the world model supplies $P(s' \mid s, a)$ for planning and constraint-checking. Conflating the two—letting the transition kernel's uncertainty masquerade as the state's certainty—is the architectural error at the root of the four failure modes.

This decoupling also reframes the safety discussion. In safety-critical settings, the danger is rarely that the world model simulates inaccurately; it is that the *current state itself is wrong* (Failure Mode 1). A perfectly accurate transition kernel, rolled forward from a corrupted s, produces confidently wrong futures. Decoupling makes the integrity of s a separable, auditable concern—one that can be subjected to hard physical-consistency checks independently of the world model's learned soft constraints.

4.5 The convergence that is already visible

The PWS direction is not empty territory. Object-addressable approaches already maintain persistent, addressable object slots and jointly predict object-centric world states and actions [10, 23, 24, 26, 27, 28]—a clear prototype of PWS at the object layer, though without task semantics. Meanwhile, event-memory WAMs compress history into task-level progress—a prototype at the task layer, though (as we argued) a state in name only.

The two compression lines, and their necessary convergence

- **Task layer:** history → task progress (but currently an orderless average)
- **Object layer:** history → object state (but currently without task indexing)

The next step is to merge the two lines: **progress defined by object states; object states indexed by task goals.**

This convergence directly answers a question we posed to the authors of a frontier WAM system: *when a user changes the goal mid-task, is the "progress" inferred under the previous goal re-estimated from scratch, or incrementally revised?* The authors' reply [54] was engineering-level and instructive: the system detects prompt changes before each action chunk, re-encodes only the changed modality, and re-injects the new conditioning into the next chunk. We note what this answer does and does not address. It handles *condition change*—a new sentence, a new goal image—but not *progress re-estimation*: nothing in the mechanism revisits what has already been accomplished in light of the new goal. Within a memory framework, this is unsurprising: because progress is not anchored to physical state, changing the goal leaves the averaged "progress" floating, and the only available move is to re-encode the conditioning and trust the next reconstruction. In a PWS framework, by contrast, the answer is structural: object states persist across goal changes; only the task-level indexing (which object states matter for the new goal) is revised. Goal revision becomes a change of *query*, not a collapse of *state*.

5 Discussion: Industrial Implications

The PWS lens clarifies several puzzles in the current embodied-AI landscape.

5.1 Why structured deployments succeed

The most credible commercial deployments to date—automotive parts handling, battery-line logistics, warehouse tote transport—share a hidden commonality: *the environment has been structured to collapse the state space*. Fixtures, jigs, conveyors, and fixed stations act as an externalized Physical World State: the cup's position is guaranteed by the jig, not tracked by the model. In our terms, these deployments

succeed because they *outsource PWS to the environment*, sidestepping the missing variable rather than solving it.

This explains a pattern that otherwise looks paradoxical: companies with modest benchmark scores achieve multi-month factory deployments, while companies with frontier world-model capabilities remain in laboratory demos. The former chose environments that do not require PWS; the latter are attempting domains (homes, unstructured manipulation) where PWS cannot be avoided.

5.2 Why world-model-native companies face a structural dilemma

A class of companies has emerged whose core thesis is that the world model itself can serve as the robot's pretrained foundation, bound to diverse embodiments with minimal in-context data. Our analysis implies this thesis contains an internal tension: the claimed advantage (data efficiency, cross-embodiment generalization) presupposes that the world model maintains a state representation good enough to support few-shot grounding. If the underlying state mechanism is mean-pooled memory retrieval, the thesis lacks an architectural foundation.

This does not mean the thesis is wrong; it means its success depends on resolving the PWS problem—whether by internalizing PWS into the world model, or by explicitly decoupling state tracking as we propose. Companies in this class that treat the problem as an engineering detail rather than an architectural gap are, in our view, underestimating the difficulty.

5.3 The evaluation gap

Current embodied-world-model benchmarks measure success rate and physical plausibility [41, 42, 44, 45, 46], but not *failure-mode distribution* [49, 48]. A 99% success rate conceals whether the 1% failures are "placed the cup slightly off-center" or "poured hot water on a person." For homes and high-hazard settings, this distinction is everything. Moreover, adversarial evaluation—deliberately inducing physical contradictions to probe state-tracking integrity—is nearly absent, in part because physical red-teaming has real costs.

The interpretability dimension compounds this. The authors of a frontier system confirm that subgoals remain implicit: control consumes the hidden states of planning tokens, and while discrete tokens can be inspected for debugging, they "cannot be regarded as fully reliable natural-language explanations" [54]. An uninterpretable subgoal representation is not merely a debugging inconvenience; it is a barrier to any form of formal verification or adversarial safety audit. One cannot red-team a subgoal one cannot read.

We regard the development of PWS-aware evaluation (testing revisability, consistency under contradiction, and goal-revision robustness) as a prerequisite for credible deployment claims in unstructured domains.

5.4 Relation to the LLM analogy

It is fashionable to analogize embodied world models to LLM-based agent operating systems, and the analogy is instructive: both involve a learned "internal world" with a lightweight execution layer (tool calls vs. motor commands). But the analogy breaks at the point that matters most here. In software agents, tool interfaces are discrete and specified; in physical agents, actions are continuous and dynamics-constrained, and errors change the world itself (Failure Mode 4). The "tool-call failure" and the "grasp failure" have fundamentally different cost structures. This is why we argue the state problem in embodied AI is not a recapitulation of the LLM memory problem but a harder, physically-entangled variant of it.

5.5 Limitations of this position

We note three limitations of our own argument. First, the PWS framework is a position, not an implemented system; the engineering cost of maintaining a persistent, revisable, physically-consistent,

counterfactually-simulable state at real-time rates is unknown and may be prohibitive. Second, hybrid architectures may blur the boundary we draw—a sufficiently structured learned memory might approximate PWS behavior without explicit state variables, and the empirical question of whether such approximations suffice remains open. Third, our industrial analysis is based on public information and anonymized cases; we may be under- or over-estimating the extent to which specific systems already incorporate PWS-like mechanisms internally.

6 Research Agenda

We close with a concrete agenda, ordered by increasing difficulty.

1. **PWS-aware evaluation.** Extend embodied-world-model benchmarks with tests for state revisability (does new evidence overwrite old belief?), consistency under contradiction (does the system detect physically impossible states?), and goal-revision robustness (does changing the goal preserve valid object-level state while updating task-level indexing?).
2. **Explicit state trackers as plug-ins.** Develop state-tracking modules that can be attached to existing WAM systems, owning the current value of s while leaving the world model to supply $P(s' \mid s, a)$. The immediate deliverable is an ablation: does an explicit state tracker reduce Failure Modes 1–4 on long-horizon benchmarks?
3. **Physical-consistency layers.** Formalize a constraint-checking layer (hand cannot be both occupied and free; cup cannot be both grasped and supported) that audits proposed states before they are committed to the tracker. This layer should be hard, non-differentiable, and independently testable.
4. **Task-object state fusion.** Implement the convergence identified in Section 4: object-level states indexed by task goals, with task progress *defined by* object states rather than averaged from history. The result should be a state representation that survives goal changes structurally rather than heuristically.
5. **Counterfactual rollout with integrity guarantees.** The hardest problem: rolling forward a state whose integrity is guaranteed by an independent tracker, through a transition kernel whose uncertainty is honestly reported. Success here would constitute, in our view, the first architecture genuinely deserving the name "embodied world model."

7 Conclusion

Return to the opening instant: the cup is placed; the robot turns to the door. Memory answers "what happened." Task descriptions answer "why." But what actually drives the robot's next action is a continuously-updated, always-present, physically-consistent representation of *what is true now*—a Physical World State.

The trajectory of World Action Models is, on our reading, already clear: from History, to Task Progress State, to Physical World State. The first step—compressing history—has been demonstrated. The second—object-level persistent state—has been prototyped. The third is the merger: *progress defined by object states; object states indexed by task goals*, all of it owned by a tracker that persists, revises, and checks consistency, while the world model does what it alone can do—score the futures.

Robots do not live on a timeline. They live in an evolving physical world. Memory cannot capture that. State can.

References

[1] D. Ha and J. Schmidhuber. World models. *arXiv preprint arXiv:1803.10122*, 2018.

[2] D. Hafner, J. Pasukonis, J. Ba, and T. Lillicrap. Mastering diverse control tasks through world models. *Nature*, 2025. arXiv:2301.04104.

[3] Y. LeCun. A path towards autonomous machine intelligence. *OpenReview preprint*, 2022.

[4] M. Assran, A. Bardes, D. Fan, Q. Garrido, R. Howes, M. Muckley, A. Rizvi, C. Roberts, K. Sinha, A. Zholus, et al. V-JEPA 2: Self-supervised video models enable understanding, prediction and planning. *arXiv preprint arXiv:2506.09985*, 2025.

[5] J. Bruce, M. D. Dennis, A. Edwards, J. Parker-Holder, Y. Shi, E. Hughes, M. Lai, A. Mavalankar, R. Steigerwald, C. Apps, et al. Genie: Generative interactive environments. In *International Conference on Machine Learning (ICML)*, 2024.

[6] N. Hansen, H. Su, and X. Wang. TD-MPC2: Scalable, robust world models for continuous control. In *International Conference on Learning Representations (ICLR)*, 2024. arXiv:2310.16828.

[7] Manifold AI. WorldScape: An efficient real-time interactive world model. Technical report, 2026. Available at https://manifoldai.cn/assets/file/WorldScape.pdf.

[8] Manifold AI. WorldScape Policy: Generalizable robotic learning via a unified world action model. Technical report, 2026. Available at https://manifoldai.cn/assets/file/WorldScapePolicy.pdf.

[9] WorldScape Policy 2.0: Empowering steerable world action modeling with reasoning-augmented memory. *arXiv preprint arXiv:2607.18840*, 2026.

[10] Y. Liu et al. OA-WAM: Object-addressable world action model for robust robot manipulation. *arXiv preprint arXiv:2605.06481*, 2026.

[11] K. Wang et al. DiM-WAM: World-action modeling with diverse historical event memory. *arXiv preprint arXiv:2606.27677*, 2026.

[12] S. Yang, J. Mu, T. Wei, C. Lu, X. Li, L. Xu, Z. Xue, Z. Yuan, D. Lin, J. Pang, and H. Xu. MemoryWAM: Efficient world action modeling with persistent memory. *arXiv preprint arXiv:2606.20562*, 2026.

[13] Mem-World: Memory-augmented action-conditioned world models for persistent robot manipulation. *arXiv preprint arXiv:2606.18960*, 2026.

[14] HiMe: Hierarchical embodied memory for long-horizon vision-language-action control. *arXiv preprint arXiv:2607.03449*, 2026.

[15] M. Torne, K. Pertsch, H. Walke, K. Vedder, S. Nair, B. Ichter, A. Z. Ren, H. Wang, J. Tang, et al. MEM: Multi-scale embodied memory for vision language action models. *arXiv preprint arXiv:2603.03596*, 2026.

[16] L. Li, Q. Zhang, Y. Luo, S. Yang, R. Wang, L. Zhang, M. Yu, Z. Gao, N. Xue, B. Zhou, X. Zhu, M. Ding, Y. Shen, and Y. Xu. Causal world modeling for robot control. In *Robotics: Science and Systems (RSS)*, 2026. arXiv:2601.21998.

[17] S. Ye, Y. Ge, K. Zheng, S. Gao, S. Yu, G. Kurian, S. Indupuru, Y. L. Tan, C. Zhu, J. Xiang, et al. World action models are zero-shot policies. *arXiv preprint arXiv:2602.15922*, 2026.

[18] J. Cen, C. Yu, H. Yuan, Y. Jiang, S. Huang, J. Guo, X. Li, Y. Song, H. Luo, F. Wang, D. Zhao, and H. Chen. WorldVLA: Towards autoregressive action world model. *arXiv preprint arXiv:2506.21539*, 2025.

[19] M. J. Kim, Y. Gao, T.-Y. Lin, Y.-C. Lin, Y. Ge, G. Lam, P. Liang, S. Song, M.-Y. Liu, C. Finn, et al. Cosmos Policy: Fine-tuning video models for visuomotor control and planning. *arXiv preprint arXiv:2601.16163*, 2026.

[20] C. Zhu, R. Yu, S. Feng, B. Burchfiel, P. Shah, and A. Gupta. Unified world models: Coupling video and action diffusion for pretraining on large robotic datasets. *arXiv preprint arXiv:2504.02792*, 2025.

[21] H. Bi, H. Tan, S. Xie, Z. Wang, S. Huang, H. Liu, R. Zhao, Y. Feng, C. Xiang, Y. Rong, et al. Motus: A unified latent action world model. *arXiv preprint arXiv:2512.13030*, 2025.

[22] G. Zhou, H. Pan, Y. LeCun, and L. Pinto. Dino-WM: World models on pre-trained visual features enable zero-shot planning. *arXiv preprint arXiv:2411.04983*, 2024.

[23] F. Locatello, D. Weissenborn, T. Unterthiner, A. Mahendran, G. Heigold, J. Uszkoreit, A. Dosovitskiy, and T. Kipf. Object-centric learning with slot attention. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2020. arXiv:2006.15055.

[24] Z. Wu, N. Dvornik, K. Greff, T. Kipf, and A. Garg. SlotFormer: Unsupervised visual dynamics simulation with object-centric models. In *International Conference on Learning Representations (ICLR)*, 2023. arXiv:2210.05861.

[25] Z. Wu, J. Hu, W. Lu, I. Gilitschenski, and A. Garg. SlotDiffusion: Object-centric generative modeling with diffusion models. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2023. arXiv:2305.11281.

[26] S. Ferraro, P. Mazzaglia, T. Verbelen, and B. Dhoedt. FOCUS: Object-centric world models for robotics manipulation. *arXiv preprint arXiv:2307.02427*, 2023.

[27] A. Chapin, E. Dellandréa, and L. Chen. STORM: Slot-based task-aware object-centric representation for robotic manipulation. *arXiv preprint arXiv:2601.20381*, 2026.

[28] Y. Jeong, J. Chun, S. Cha, and T. Kim. Object-centric world model for language-guided manipulation. *arXiv preprint arXiv:2503.06170*, 2025.

[29] B. Zitkovich, T. Yu, S. Xu, P. Xu, T. Xiao, F. Xia, J. Wu, P. Wohlhart, S. Welker, A. Wahid, et al. RT-2: Vision-language-action models transfer web knowledge to robotic control. In *Conference on Robot Learning (CoRL)*, 2023. arXiv:2307.15818.

[30] M. J. Kim, K. Pertsch, S. Karamcheti, T. Xiao, A. Balakrishna, S. Nair, R. Rafailov, E. Foster, G. Lam, P. Sanketi, et al. OpenVLA: An open-source vision-language-action model. *arXiv preprint arXiv:2406.09246*, 2024.

[31] K. Black, N. Brown, D. Driess, A. Esmail, M. Equi, C. Finn, N. Fusai, L. Groom, K. Hausman, B. Ichter, et al. π_0: A vision-language-action flow model for general robot control. In *Robotics: Science and Systems (RSS)*, 2025. arXiv:2410.24164.

[32] Physical Intelligence, A. Amin, R. Aniceto, A. Balakrishna, K. Black, K. Conley, G. Connors, J. Darpinian, K. Dhabalia, J. DiCarlo, et al. $\pi_{*0.6}$: A VLA that learns from experience. *arXiv preprint arXiv:2511.14759*, 2025.

[33] NVIDIA GEAR Team. GR00T N1.5: An improved open foundation model for generalist humanoid robots. NVIDIA Research Blog, 2025.

[34] C. Chi, Z. Xu, S. Feng, E. Cousineau, Y. Du, B. Burchfiel, R. Tedrake, and S. Song. Diffusion Policy: Visuomotor policy learning via action diffusion. In *Robotics: Science and Systems (RSS)*, 2023. arXiv:2303.04137.

[35] Octo Model Team, D. Ghosh, H. Walke, K. Pertsch, K. Black, O. Mees, S. Dasari, J. Hejna, T. Kreiman, C. Xu, et al. Octo: An open-source generalist robot policy. In *Robotics: Science and Systems (RSS)*, 2024. arXiv:2405.12213.

[36] L. P. Kaelbling, M. L. Littman, and A. R. Cassandra. Planning and acting in partially observable stochastic domains. *Artificial Intelligence*, 101(1–2):99–134, 1998.

[37] L. P. Kaelbling and T. Lozano-Pérez. Integrated task and motion planning in belief space. *The International Journal of Robotics Research*, 32(9–10):1194–1227, 2013.

[38] C. R. Garrett, C. Paxton, T. Lozano-Pérez, L. P. Kaelbling, and D. Fox. Online replanning in belief space for partially observable task and motion problems. In *IEEE International Conference on Robotics and Automation (ICRA)*, pp. 5678–5684, 2020.

[39] C. R. Garrett, R. Chitnis, R. Holladay, B. Kim, T. Silver, L. P. Kaelbling, and T. Lozano-Pérez. Integrated task and motion planning. *Annual Review of Control, Robotics, and Autonomous Systems*, 4:265–293, 2021.

[40] R. Chitnis, T. Silver, J. B. Tenenbaum, T. Lozano-Pérez, and L. P. Kaelbling. Learning neuro-symbolic relational transition models for bilevel planning. In *IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)*, 2022.

[41] H. Duan, H.-X. Yu, S. Chen, L. Fei-Fei, and J. Wu. WorldScore: A unified evaluation benchmark for world generation. In *IEEE/CVF International Conference on Computer Vision (ICCV)*, pp. 27713–27724, 2025. arXiv:2504.00983.

[42] Y. Shang, Z. Li, Y. Ma, W. Su, X. Jin, Z. Wang, L. Jin, X. Zhang, Y. Tang, H. Su, et al. WorldArena: A unified benchmark for evaluating perception and functional utility of embodied world models. *arXiv preprint arXiv:2602.08971*, 2026.

[43] Y. Mu, T. Chen, Z. Chen, S. Peng, Z. Lan, Z. Gao, Z. Liang, Q. Yu, Y. Zou, M. Xu, et al. RoboTwin: Dual-arm robot benchmark with generative digital twins. In *IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, pp. 27649–27660, 2025.

[44] T. Chen, Z. Chen, B. Chen, Z. Cai, Y. Liu, Q. Liang, Z. Li, X. Lin, Y. Ge, Z. Gu, et al. RoboTwin 2.0: A scalable data generator and benchmark with strong domain randomization for robust bimanual robotic manipulation. *arXiv preprint arXiv:2506.18088*, 2025.

[45] B. Liu, Y. Zhu, C. Gao, Y. Feng, Q. Liu, Y. Zhu, and P. Stone. LIBERO: Benchmarking knowledge transfer for lifelong robot learning. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2023. arXiv:2306.03310.

[46] X. Li, K. Hsu, J. Gu, K. Pertsch, O. Mees, H. R. Walke, C. Fu, I. Lunawat, I. Sieh, S. Kirmani, et al. Evaluating real-world robot manipulation policies in simulation. In *Conference on Robot Learning (CoRL)*, 2024. arXiv:2405.05941.

[47] O. Mees, L. Hermann, E. Rosete-Beas, and W. Burgard. CALVIN: A benchmark for language-conditioned policy learning for long-horizon robot manipulation tasks. *IEEE Robotics and Automation Letters*, 7(3):7327–7334, 2022.

[48] D. Li, Y. Fang, Y. Chen, S. Yang, S. Cao, J. Wong, M. Luo, X. Wang, H. Yin, J. E. Gonzalez, et al. WorldModelBench: Judging video generation models as world models. *arXiv preprint arXiv:2502.20694*, 2025.

[49] Z. Ma, M. Liufu, and G. Gkioxari. Out of sight, out of mind? Evaluating state evolution in video world models. *arXiv preprint arXiv:2603.13215*, 2026.

[50] Y. Lipman, R. T. Q. Chen, H. Ben-Hamu, M. Nickel, and M. Le. Flow matching for generative modeling. In *International Conference on Learning Representations (ICLR)*, 2023. arXiv:2210.02747.

[51] Open X-Embodiment Collaboration, A. O'Neill, A. Rehman, A. Gupta, A. Maddukuri, A. Gupta, A. Padalkar, A. Lee, A. Pooley, A. Gupta, et al. Open X-Embodiment: Robotic learning datasets and RT-X models. *arXiv preprint arXiv:2310.08864*, 2023.

[52] A. Khazatsky, K. Pertsch, S. Nair, A. Balakrishna, S. Dasari, S. Karamcheti, S. Nasiriany, M. K. Srirama, L. Y. Chen, K. Ellis, et al. DROID: A large-scale in-the-wild robot manipulation dataset. In *Robotics: Science and Systems (RSS)*, 2024. arXiv:2403.12945.

[53] World model for robot learning: A comprehensive survey. *arXiv preprint arXiv:2604.16592*, 2026.

[54] Authors of a frontier WAM system. Private communication (public comment-thread exchange with the author of this paper), July 2026. Specific implementation details confirmed: linear growth of memory with history; training-time event boundaries from embodiment signals vs. online cosine-change selection; implicit (hidden-state) subgoals; chunk-level prompt re-encoding for interactive goal changes.